TÉCNICAS DE HACKING

Práctica 2: Reconocimiento Activo

Juan José Ibáñez

Universidad Europea de Madrid

17/04/2026

Resumen

En esta práctica se estudia el reconocimiento activo de redes, una fase esencial en cualquier auditoría de ciberseguridad. A diferencia del reconocimiento pasivo, el reconocimiento activo implica el envío deliberado de paquetes de red a los sistemas objetivo para deducir qué equipos están encendidos y qué servicios ofrecen — en términos no técnicos, se llama a la puerta de cada casa del vecindario para ver quién está en casa. Se ha implementado en Python la función `craft_discovery_pkts` usando la librería Scapy, que construye paquetes de descubrimiento basados en los protocolos ICMP Timestamp, TCP ACK y UDP. Además, se analiza el comportamiento por defecto de Nmap para comprender qué mensajes envía y cómo clasifica el estado de los puertos. Todas las pruebas se realizaron sobre contenedores Docker en cumplimiento de las restricciones legales y éticas.

Índice

1. 2
2. Introducción 3
3. Desarrollo 3
    3.1. Descubrimiento de hosts con Scapy 3
        3.1.1. La librería Scapy 3
        3.1.2. Protocolos de descubrimiento 3
        3.1.3. Implementación: función `craft_discovery_pkts` 4
    3.2. Comportamiento por defecto de Nmap y estado de puertos 4
        3.2.1. Estado de un puerto 4
        3.2.2. Estímulos y respuestas por estado 4
        3.2.3. Comportamiento por defecto de Nmap 4
4. Resultados 5
    4.1. Entorno de simulación 5
    4.2. Parte 1: Descubrimiento de hosts con Scapy 5
    4.3. Parte 2: Comportamiento por defecto de Nmap 7
5. Conclusiones finales del proyecto P2 9
Bibliografía 9

1.

2. Introducción

El reconocimiento activo es la segunda fase de la metodología de auditoría de seguridad ofensiva, inmediatamente posterior al reconocimiento pasivo [1]. Mientras el reconocimiento pasivo recopila información sin interactuar con el objetivo, el reconocimiento activo implica el envío deliberado de tráfico de red para provocar respuestas que revelen el estado y configuración de los sistemas.

Esta fase permite determinar qué equipos están activos en la red y qué puertos tienen servicios escuchando (abiertos), cuáles los rechazan (cerrados) y cuáles están protegidos por filtrado (filtrados). Esta información es imprescindible para planificar las fases posteriores de la auditoría.

En esta práctica se abordan dos bloques. El primero implementa un descubridor de hosts usando Scapy [2], [3], empleando paquetes ICMP Timestamp [4], TCP ACK [5] y UDP [6]. El segundo analiza el comportamiento por defecto de Nmap [7], [8], la herramienta de escaneo más usada en la industria. Todo el trabajo experimental se realizó sobre un entorno Docker en Kali Linux.

3. Desarrollo

3.1. Descubrimiento de hosts con Scapy

3.1.1. La librería Scapy

Scapy es una librería de manipulación de paquetes escrita en Python que permite construir paquetes de red arbitrarios en cada capa del modelo OSI, enviarlos y analizar las respuestas [2]. La función central usada es `sr()` (*send and receive*): envía paquetes y devuelve dos listas, los que obtuvieron respuesta y los que no. Esta distinción es la base del descubrimiento de hosts.

3.1.2. Protocolos de descubrimiento

ICMP Timestamp Request (tipo 13 / tipo 14)

El protocolo ICMP [4] incluye el tipo 13 (*Timestamp Request*): el emisor solicita al receptor su hora actual. Si el host está activo, responde con tipo 14 (*Timestamp Reply*). La ventaja frente al ping (tipo 8) es que algunos cortafuegos bloquean Echo Request pero permiten Timestamp, ampliando la cobertura.

TCP ACK Scan

El TCP ACK Scan envía un segmento TCP con el flag ACK activado sin haber iniciado ninguna conexión previa [5]. El host responde con RST porque el sistema operativo no reconoce la sesión TCP a la que pertenecería ese ACK, independientemente de si el puerto destino está abierto o cerrado. Muchos cortafuegos permiten el tráfico ACK pensando que pertenece a una sesión legítima, lo que hace esta técnica eficaz.

UDP Discovery

Al enviar un datagrama UDP a un puerto sin servicio en un host activo, el sistema genera ICMP tipo 3 código 3 (*Port Unreachable*) [4], confirmando la presencia del host [6]. Si el puerto tiene un servicio escuchando, puede que no haya respuesta, haciendo la interpretación más ambigua.

3.1.3. Implementación: función `craft_discovery_pkts`

La función recibe los siguientes argumentos:

- `protocols`: lista de hasta 3 protocolos (`"ICMP"`, `"TCP"`, `"UDP"`) o un solo string.
- `ip_range`: IP o rango en formato Scapy (p.ej. `"172.20.0.0/24"`).
- `pkt_count` *(opcional)*: diccionario `{protocolo: n_paquetes}`. Por defecto 1.
- `port` *(opcional)*: puerto TCP/UDP destino. Por defecto 80.

La función itera sobre cada protocolo y construye el paquete apilando capas con el operador `/` de Scapy: `IP(dst=...) / ICMP(type=13)` para ICMP Timestamp, `IP(dst=...) / TCP(dport=port, flags="A")` para TCP ACK y `IP(dst=...) / UDP(dport=port)` para UDP. Todos los paquetes se acumulan en una lista y se retornan. Para el envío y detección se usa `sr()`, clasificando las respuestas en respondidas y no respondidas.

3.2. Comportamiento por defecto de Nmap y estado de puertos

3.2.1. Estado de un puerto

Nmap [7] clasifica los puertos en tres estados según el estímulo y la respuesta:

Abierto (*open*): Hay un servicio escuchando. El sistema acepta conexiones.

Cerrado (*closed*): El host está activo pero no hay servicio en ese puerto. El sistema responde indicando el rechazo.

Filtrado (*filtered*): Un cortafuegos intercepta los paquetes antes de llegar al puerto. No es posible determinar si hay servicio.

3.2.2. Estímulos y respuestas por estado

Estado	Estímulo enviado	Respuesta recibida
Abierto	TCP SYN al puerto	TCP SYN+ACK
Cerrado	TCP SYN al puerto	TCP RST
Filtrado	TCP SYN al puerto	Sin respuesta o ICMP Unreachable

Tabla 1: Estados de puerto según estímulo y respuesta en TCP SYN scan.

3.2.3. Comportamiento por defecto de Nmap

Al ejecutar `nmap <objetivo>` como root, Nmap realiza dos fases [8]:

Fase 1 — Descubrimiento de host: Envía cuatro sondas: ICMP Echo Request, TCP SYN al 443, TCP ACK al 80 e ICMP Timestamp. Si el host no responde a ninguna, no se escanea.

Fase 2 — Escaneo de puertos: TCP SYN Scan (`-sS`) sobre los 1.000 puertos más comunes. Al recibir SYN+ACK (abierto), Nmap responde con RST para no completar la conexión — de

ahí el nombre *half-open* o *stealth scan*. RST indica cerrado; sin respuesta o ICMP Unreachable indica filtrado.

4. Resultados

4.1. Entorno de simulación

Las pruebas se realizaron con dos contenedores Docker en Kali Linux:

- `172.20.0.1` — Gateway de la red Docker (host Kali).
- `172.20.0.2` — Contenedor `nginx_p2`: servidor web Nginx, puerto 80 abierto.
- `172.20.0.3` — Contenedor `ssh_p2`: servidor OpenSSH, puerto 22 abierto.
- `172.20.0.200` — IP sin asignar, usada como control negativo.

4.2. Parte 1: Descubrimiento de hosts con Scapy

La función `craft_discovery_pkts` se ejecutó sobre el rango `172.20.0.0/24` con los tres protocolos. Los resultados se presentan en la Tabla 2.

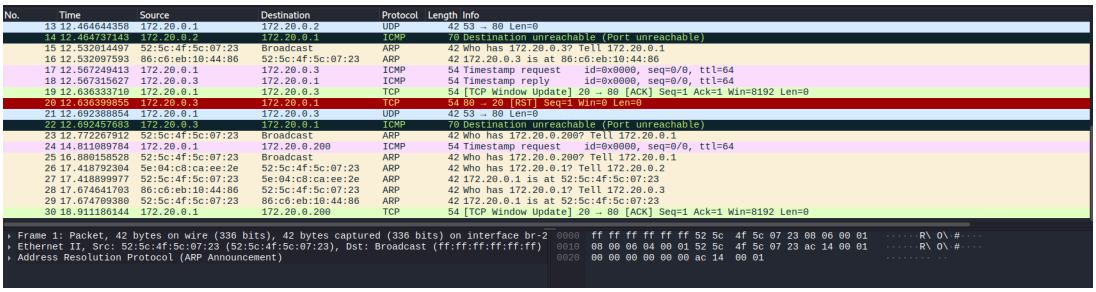
IP destino	Protocolo	Respuesta obtenida	Estado
172.20.0.1	ICMP type=13	ICMP type=14 (Timestamp Reply)	**Activo**
172.20.0.1	TCP ACK	TCP RST	**Activo**
172.20.0.1	UDP p:80	ICMP Port Unreachable (t=3, c=3)	**Activo**
172.20.0.2	ICMP type=13	ICMP type=14 (Timestamp Reply)	**Activo**
172.20.0.2	TCP ACK	TCP RST	**Activo**
172.20.0.2	UDP p:80	Sin respuesta (Nginx escucha p:80)	Indeterminado
172.20.0.3	ICMP type=13	ICMP type=14 (Timestamp Reply)	**Activo**
172.20.0.3	TCP ACK	TCP RST	**Activo**
172.20.0.3	UDP p:80	ICMP Port Unreachable (t=3, c=3)	**Activo**
172.20.0.200	ICMP type=13	Sin respuesta (timeout 2s)	Inactivo
172.20.0.200	TCP ACK	Sin respuesta (timeout 2s)	Inactivo
172.20.0.200	UDP p:80	Sin respuesta (timeout 2s)	Inactivo

Tabla 2: Resultados del descubrimiento de hosts con `craft_discovery_pkts`.

La combinación de tres protocolos aumenta la robustez: en `172.20.0.2` el UDP no generó respuesta porque Nginx estaba escuchando en el 80, pero ICMP y TCP ACK sí confirmaron el host activo. El host inactivo `172.20.0.200` no generó respuesta en ningún protocolo.

```
┌──(w0lfhack05㉿kali)-[~/Hacking/P2_Reconocimiento_activo]
└─$ sudo python3 src/host_discovery.py
═══════════════════════════════════════════════════════
     Descubrimiento de Hosts con Scapy
     Práctica 2 - Reconocimiento Activo
═══════════════════════════════════════════════════════

[*] Targets: ['172.20.0.1', '172.20.0.2', '172.20.0.3']
[*] Protocolos: ['ICMP', 'TCP', 'UDP']
[*] Puerto L4: 80

[*] Construyendo paquetes para el rango: 172.20.0.1
[*] Total de paquetes construidos: 3
[*] Enviando y esperando respuestas (timeout=2s)...

[-] Sin respuesta de 172.20.0.1 con protocolo ICMP
[-] Sin respuesta de 172.20.0.1 con protocolo TCP
[-] Sin respuesta de 172.20.0.1 con protocolo UDP
[*] Construyendo paquetes para el rango: 172.20.0.2
[*] Total de paquetes construidos: 3
[*] Enviando y esperando respuestas (timeout=2s)...

[+] Host activo detectado: 172.20.0.2
[*] Construyendo paquetes para el rango: 172.20.0.3
[*] Total de paquetes construidos: 3
[*] Enviando y esperando respuestas (timeout=2s)...

[+] Host activo detectado: 172.20.0.3

═══════════════════════════════════════════════════════
  Resumen: 2 host(s) activo(s)
═══════════════════════════════════════════════════════
  → 172.20.0.2
  → 172.20.0.3

[*] Probando con una IP que no debería estar activa...
[-] 172.20.0.200 no responde a ICMP → Host inactivo
[-] 172.20.0.200 no responde a TCP → Host inactivo

┌──(w0lfhack05㉿kali)-[~/Hacking/P2_Reconocimiento_activo]
└─$
```

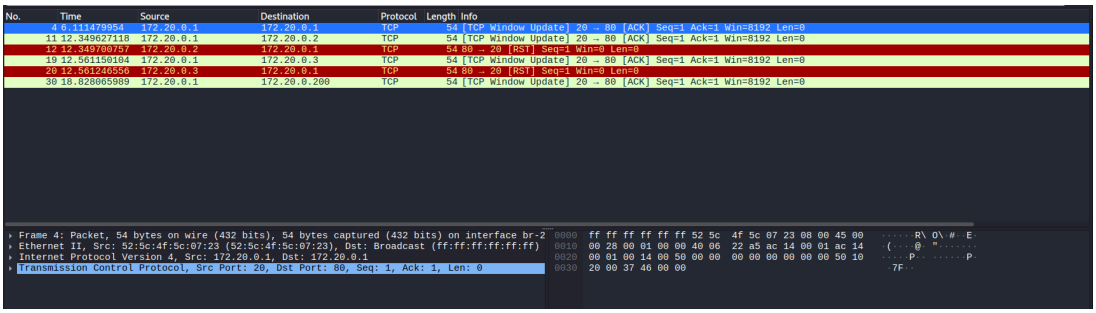
Figura 1: Salida del script `host_discovery.py` ejecutado sobre la red `172.20.0.0/24`. Se aprecian los hosts que respondieron y el host inactivo sin respuesta.



No.	Time	Source	Destination	Protocol	Length	Info
13	12.464644358	172.20.0.1	172.20.0.2	UDP	42	53 → 80 Len=0
14	12.464737143	172.20.0.2	172.20.0.1	ICMP	70	Destination unreachable (Port unreachable)
15	12.532014497	52:5c:4f:5c:07:23	Broadcast	ARP	42	Who has 172.20.0.3? Tell 172.20.0.1
16	12.532097593	86:c6:eb:10:44:86	52:5c:4f:5c:07:23	ARP	42	172.20.0.3 is at 86:c6:eb:10:44:86
17	12.567249413	172.20.0.1	172.20.0.3	ICMP	54	Timestamp request id=0x0000, seq=0/0, ttl=64
18	12.567315627	172.20.0.3	172.20.0.1	ICMP	54	Timestamp reply id=0x0000, seq=0/0, ttl=64
19	12.636333710	172.20.0.1	172.20.0.3	TCP	54	[TCP Window Update] 20 → 80 [ACK] Seq=1 Ack=1 Win=8192 Len=0
20	12.636399855	172.20.0.3	172.20.0.1	TCP	54	80 → 20 [RST] Seq=1 Win=0 Len=0
21	12.692388854	172.20.0.1	172.20.0.3	UDP	42	53 → 80 Len=0
22	12.692457683	172.20.0.3	172.20.0.1	ICMP	70	Destination unreachable (Port unreachable)
23	12.772267912	52:5c:4f:5c:07:23	Broadcast	ARP	42	Who has 172.20.0.200? Tell 172.20.0.1
24	14.811089784	172.20.0.1	172.20.0.200	ICMP	54	Timestamp request id=0x0000, seq=0/0, ttl=64
25	16.880158528	52:5c:4f:5c:07:23	Broadcast	ARP	42	Who has 172.20.0.200? Tell 172.20.0.1
26	17.418792304	5e:04:c8:ca:ee:2e	52:5c:4f:5c:07:23	ARP	42	Who has 172.20.0.1? Tell 172.20.0.2
27	17.418899977	52:5c:4f:5c:07:23	5e:04:c8:ca:ee:2e	ARP	42	172.20.0.1 is at 52:5c:4f:5c:07:23
28	17.674641703	86:c6:eb:10:44:86	52:5c:4f:5c:07:23	ARP	42	Who has 172.20.0.1? Tell 172.20.0.3
29	17.674709380	52:5c:4f:5c:07:23	86:c6:eb:10:44:86	ARP	42	172.20.0.1 is at 52:5c:4f:5c:07:23
30	18.911186144	172.20.0.1	172.20.0.200	TCP	54	[TCP Window Update] 20 → 80 [ACK] Seq=1 Ack=1 Win=8192 Len=0

```
▸ Frame 1: Packet, 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface br-2
▸ Ethernet II, Src: 52:5c:4f:5c:07:23 (52:5c:4f:5c:07:23), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
▸ Address Resolution Protocol (ARP Announcement)

0000  ff ff ff ff ff ff 52 5c  4f 5c 07 23 08 06 00 01   ····· R\ O\ #····
0010  08 00 06 04 00 01 52 5c  4f 5c 07 23 ac 14 00 01   ····· R\ O\ #····
0020  00 00 00 00 00 00 ac 14  00 01                     ····· ··
```

Figura 2: Captura Wireshark del intercambio ICMP Timestamp. Se observa la petición (type=13) desde Kali y la respuesta (type=14) del host activo.



No.	Time	Source	Destination	Protocol	Length	Info
4	6.111479954	172.20.0.1	172.20.0.1	TCP	54	[TCP Window Update] 20 → 80 [ACK] Seq=1 Ack=1 Win=8192 Len=0
11	12.349627118	172.20.0.1	172.20.0.2	TCP	54	[TCP Window Update] 20 → 80 [ACK] Seq=1 Ack=1 Win=8192 Len=0
12	12.349703767	172.20.0.2	172.20.0.1	TCP	54	80 → 20 [RST] Seq=1 Win=0 Len=0
19	12.561150104	172.20.0.1	172.20.0.3	TCP	54	[TCP Window Update] 20 → 80 [ACK] Seq=1 Ack=1 Win=8192 Len=0
20	12.561246556	172.20.0.3	172.20.0.1	TCP	54	80 → 20 [RST] Seq=1 Win=0 Len=0
30	18.828065989	172.20.0.1	172.20.0.200	TCP	54	[TCP Window Update] 20 → 80 [ACK] Seq=1 Ack=1 Win=8192 Len=0

```
▸ Frame 4: Packet, 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface br-2
▸ Ethernet II, Src: 52:5c:4f:5c:07:23 (52:5c:4f:5c:07:23), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
▸ Internet Protocol Version 4, Src: 172.20.0.1, Dst: 172.20.0.1
▸ Transmission Control Protocol, Src Port: 20, Dst Port: 80, Seq: 1, Ack: 1, Len: 0

0000  ff ff ff ff ff ff 52 5c  4f 5c 07 23 08 00 45 00   ····· R\ O\ #··E·
0010  00 28 00 01 00 00 40 06  22 a5 ac 14 00 01 ac 14   ·(····@· "·····
0020  00 01 00 14 00 50 00 00  00 00 00 00 00 00 50 10   ····· P····· P·
0030  20 00 37 46 00 00                                   ··7F··
```

Figura 3: Captura Wireshark del TCP ACK Scan. El paquete ACK enviado provoca una respuesta RST del host activo, confirmando su presencia.

4.3. Parte 2: Comportamiento por defecto de Nmap

Para analizar Nmap se ejecutó `sudo nmap 172.20.0.2` mientras se capturaba el tráfico con Wireshark. Se observaron dos fases diferenciadas.

Fase 1 — Descubrimiento: Nmap envió cuatro sondas (ICMP Echo, SYN/443, ACK/80, ICMP Timestamp) antes de iniciar el escaneo de puertos. El host respondió.

Fase 2 — Escaneo: 1.000 paquetes TCP SYN a los puertos más comunes. Puerto 80 respondió con SYN+ACK (abierto). El resto respondió con RST (cerrado) o no respondió (filtrado).

```
┌──(w0lfhack05㉿kali)-[~/Hacking/P2_Reconocimiento_activo]
└─$ sudo nmap -Pn 172.20.0.2
sudo nmap -Pn 172.20.0.3
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-17 18:47 +0200
Nmap scan report for 172.20.0.2
Host is up (0.000014s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE SERVICE
80/tcp open  http
MAC Address: 5E:04:C8:CA:EE:2E (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.84 seconds
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-17 18:47 +0200
Nmap scan report for 172.20.0.3
Host is up (0.000019s latency).
Not shown: 999 closed tcp ports (reset)
PORT     STATE SERVICE
2222/tcp open  EtherNetIP-1
MAC Address: 86:C6:EB:10:44:86 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.79 seconds

┌──(w0lfhack05㉿kali)-[~/Hacking/P2_Reconocimiento_activo]
└─$
```

Figura 4: Salida de `sudo nmap 172.20.0.2`. Se identifican el puerto 80 abierto, el resto cerrados, y el resumen del escaneo.

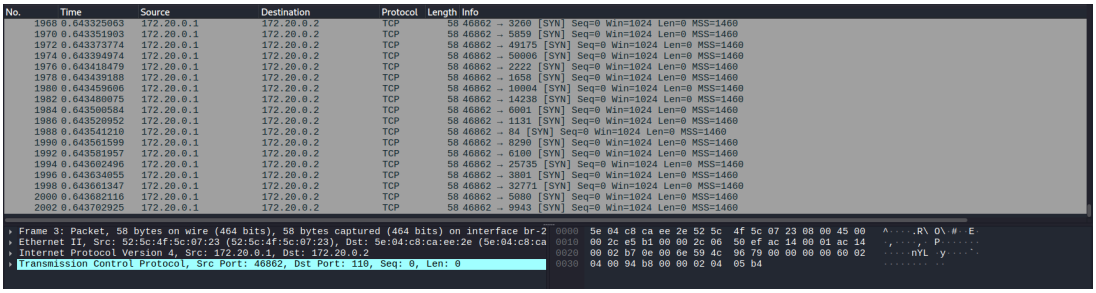


Figura 5: Captura Wireshark del tráfico generado por Nmap. Se distinguen los paquetes SYN del escaneo y las respuestas SYN+ACK (puerto abierto) y RST (cerrado).

Host	Puerto	Servicio	Estímulo	Respuesta	Estado
172.20.0.2	80/tcp	http	TCP SYN	TCP SYN+ACK	**Abierto**
172.20.0.2	443/tcp	https	TCP SYN	TCP RST	Cerrado
172.20.0.2	22/tcp	ssh	TCP SYN	TCP RST	Cerrado
172.20.0.3	22/tcp	ssh	TCP SYN	TCP SYN+ACK	**Abierto**
172.20.0.3	80/tcp	http	TCP SYN	TCP RST	Cerrado
172.20.0.3	443/tcp	https	TCP SYN	TCP RST	Cerrado

Tabla 3: Resultados del escaneo Nmap sobre los contenedores Docker.

Parámetro	Valor
Tipo de escaneo por defecto (root)	TCP SYN Scan (`-sS`)
Puertos escaneados	1.000 más comunes
Paquetes SYN enviados (mínimo)	1.000
Reintentos sin respuesta	2 (`--max-retries`)
Sondas de descubrimiento de host	4 (ICMP Echo, SYN/443, ACK/80, ICMP TS)
Velocidad por defecto	T3 (normal)
Resolución DNS inversa	Sí (hosts activos)

Tabla 4: Parámetros del comportamiento por defecto de Nmap.

5. Conclusiones finales del proyecto P2

A lo largo de la práctica se ha podido verificar que el reconocimiento activo requiere combinar múltiples técnicas para maximizar la cobertura. El ICMP Timestamp ofrece una alternativa al ping clásico menos filtrada por cortafuegos. El TCP ACK aprovecha el comportamiento estándar del stack TCP [5] para detectar hosts incluso cuando los puertos están cerrados. El UDP es útil cuando el servicio no escucha en el puerto de destino, aunque la ausencia de respuesta es ambigua.

Respecto a Nmap [7], se confirmó que el TCP SYN Scan por defecto es eficiente al no completar el *three-way handshake*, dejando menos rastros en los logs. La clasificación en tres estados (abierto, cerrado, filtrado) responde directamente a la presencia o ausencia de respuesta y al tipo de flag TCP observado.

Bibliografía

[1] C. McNab, *Network Security Assessment: Know Your Network*, 3.ª ed. O'Reilly Media, 2017.

[2] P. Biondi, «Scapy: A Powerful Interactive Packet Manipulation Program». Accedido: 17 de abril de 2026. [En línea]. Disponible en: https://scapy.net/

[3] Scapy Project, «Scapy Documentation». Accedido: 17 de abril de 2026. [En línea]. Disponible en: https://scapy.readthedocs.io/

[4] J. Postel, «Internet Control Message Protocol», RFC 792, 1981. [En línea]. Disponible en: https://www.rfc-editor.org/rfc/rfc792

[5] J. Postel, «Transmission Control Protocol», RFC 793, 1981. [En línea]. Disponible en: https://www.rfc-editor.org/rfc/rfc793

[6] J. Postel, «User Datagram Protocol», RFC 768, 1980. [En línea]. Disponible en: https://www.rfc-editor.org/rfc/rfc768

[7] G. F. Lyon, *Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning*. Nmap Project, 2009. [En línea]. Disponible en: https://nmap.org/book/

[8] Nmap Project, «Nmap Reference Guide». Accedido: 17 de abril de 2026. [En línea]. Disponible en: https://nmap.org/book/man.html